LAB 04 — Text Classification (IMDb sentiment)

Bài toán: phân loại cảm xúc review phim (positive / negative) trên **IMDb** (25 000 train / 25 000 test, cân bằng 50/50). Pipeline: `raw text → clean → TF-IDF | Average Word2Vec → Logistic Regression / Linear SVM → evaluation → error analysis`.

Cấu trúc thư mục

File	Nội dung
`report.pdf`	**Bản tổng hợp**: ghép PDF của README, calculations, prediction, experiment.ipynb, error_analysis, reflection (theo thứ tự đọc)
`calculations.md`, `calculations.pdf`	4 bài tính toán (sigmoid, threshold, confusion matrix, accuracy trap): `calculations.pdf` là bài làm viết tay (bản scan), `calculations.md` là bản gõ lại
`prediction.md`	5 dự đoán + dự đoán chính (TF-IDF vs Avg Word2Vec), dựa trên đặc tính của representation; sinh viên chọn phương án và mức tự tin
`experiment.ipynb`	Toàn bộ thí nghiệm, đã chạy sẵn, có output
`preprocessing.py`	`clean_text`, `tokenize`, `make_splits`, `average_embedding`, `load_imdb`
`results.csv`	Mọi kết quả (experiment, representation, classifier, split, accuracy, precision, recall, F1, thời gian)
`confusion_matrix.png`	Confusion matrix 3 model trên test
`error_analysis.md`	3 mẫu đúng + 4 mẫu sai, phân loại nguyên nhân
`reflection.md`	Đối chiếu prediction, giải thích khác biệt representation, hướng tới neural NLP
`class_distribution.png`, `top_features.png`	Hình bổ trợ

Thiết lập thí nghiệm

- **Split:** train gốc (25 000, loại 96 review trùng → 24 904) chia stratified 80/20 thành train 19 923 / validation 4 981 (seed 42). Test = tập test chính thức của IMDb (25 000), **không** dùng để fit hay chọn siêu tham số.
- **Chống leakage:** TfidfVectorizer, StandardScaler, Word2Vec và classifier chỉ được `fit` trên train; `C` chọn theo F1 trên validation.
- **TF-IDF:** `min_df=2`, `sublinear_tf=True`, vocab 40 790 (unigram) / 364 040 (uni+bigram).
- **Word2Vec:** gensim CBOW, 100 chiều, window 5, 10 epochs, train chỉ trên văn bản train (không dùng pretrained); document vector = trung bình, chuẩn hóa bằng StandardScaler.

Kết quả chính (test set)

Representation	Classifier	Accuracy	Precision	Recall	F1	Thời gian train
TF-IDF (unigram)	Logistic Regression	0.8814	0.8842	0.8778	0.8810	3.1 s
Avg Word2Vec	Logistic Regression	0.8426	0.8435	0.8413	0.8424	51.9 s
TF-IDF (unigram)	Linear SVM	0.8886	0.8862	0.8917	0.8889	2.2 s
TF-IDF (uni + bigram)	Logistic Regression	**0.9022**	0.9019	0.9027	**0.9023**	9.3 s

Các phát hiện chính:

1. **TF-IDF > Avg Word2Vec** (+3.9 điểm accuracy) trên IMDb: review dài, tín hiệu cảm xúc nằm ở từ cụ thể; averaging làm loãng tín hiệu và mất thứ tự.
2. **Bigram** tăng F1 +2.1 điểm (0.8810 → 0.9023) nhờ bắt được *not good / not bad* nhưng vocab tăng 8.9×, thời gian train gấp 3.0×.
3. **Bỏ stopwords làm giảm** val F1 (0.8920 → 0.8841) vì xóa mất *not, no, never, very*.
4. **Word order (Exp 4):** cos(avg("this movie is good"), avg("...is not good")) = 0.948 → average embedding gần như không phân biệt hai câu.
5. **Accuracy trap:** trên tập 950 neg / 50 pos, model "luôn đoán negative" có accuracy 0.95 nhưng F1 = 0.

Cách chạy lại

```
pip install scikit-learn gensim datasets pandas matplotlib seaborn nbformat nbconvert
ipykernel
cd lab04
jupyter nbconvert --to notebook --execute --inplace experiment.ipynb   # ~2-3 phút, cần
internet để tải IMDb
```

AI assistance statement

- **Tool:** Claude Code (Claude Sonnet 5.5)
- **Purpose:**
- Viết và chạy toàn bộ code thí nghiệm (`experiment.ipynb` , `preprocessing.py`), tạo biểu đồ, tạo `results.csv` .
- Gõ lại bài tính toán viết tay của người làm vào `calculations.md` v
- Trình bày và ghép các file thành `report.pdf` .
- **Generated (do AI):** code, biểu đồ, `results.csv` , `report.pdf` ,
- **Student (do sinh viên):** bài tính toán viết tay (`calculations.pdf`); chọn phương án và mức tự tin cho từng dự đoán trong `prediction.md` ; chọn nguyên nhân cho từng mẫu và kết luận về confusion matrix trong `error_analysis.md` ; chọn các ý trong `reflection.md` . AI chỉ ghi lại các lựa chọn đó.
- **Verified:** code đã chạy thành công đầu-cuối (kết quả trong `experiment.ipynb` / `results.csv`); các kết quả tính toán (z, σ(z), precision, recall, F1) đã được đối chiếu bằng code với tính tay; các số liệu trích trong `error_analysis.md` và `reflection.md` lấy từ output của notebook.

LAB 04 — Bài tập tính toán

Bản gõ lại từ bài làm viết tay (`calculations.pdf`).

Bài 1 — Binary classification

Cho $\mathbf{x} = [1, 2, 0]$, $\mathbf{w} = [0.5, -0.2, 0.4]$, $b = 0.1$.

Tính $z = \mathbf{w}^T\mathbf{x} + b$:

- $\mathbf{w}^T\mathbf{x} = 0.5 \times 1 + (-0.2) \times 2 + 0.4 \times 0 = 0.5 + (-0.4) = 0.1$
- $z = 0.1 + 0.1 = \mathbf{0.2}$

Dùng sigmoid:

$\sigma(z) = 1 / (1 + e^{-z}) = 1 / (1 + e^{-0.2}) \approx 0.5498$

Vậy $P(y = 1 \mid x) = \sigma(z) \approx \mathbf{0.5498}$.

Bài 2 — Prediction threshold

$P(y = 1 \mid x) = 0.72$; $\tau_1 = 0.5$; $\tau_2 = 0.8$.

- $\tau_1 = 0.5$: $0.72 > \tau_1$ nên dự đoán $\hat{y} = \mathbf{1}$.
- $\tau_2 = 0.8$: $0.72 < \tau_2$ nên dự đoán $\hat{y} = \mathbf{0}$.

Ảnh hưởng của threshold:

- **Tăng τ:** dự đoán positive trở nên khó hơn. FP giảm (precision thường tăng), FN tăng (vì có nhiều mẫu có xác suất cao nhưng không vượt qua τ nên bị dự đoán là negative), recall giảm.
- **Giảm τ:** model dự đoán positive dễ hơn nên recall tăng, FP tăng, precision giảm.

⇒ τ tăng thì precision (thường) tăng, recall giảm; τ giảm thì recall tăng, precision giảm.

Bài 3 — Confusion matrix

TP = 80, FN = 20, FP = 10, TN = 90.

- Precision = TP / (TP + FP) = 80 / (80 + 10) = 8/9 ≈ **0.889**
- Recall = TP / (TP + FN) = 80 / (80 + 20) = **0.8**
- F1 = 2PR / (P + R) = 2 × (8/9) × 0.8 / (0.8 + 8/9) = 1.4222 / 1.6889 ≈ **0.8421**

Trả lời (bài toán phát hiện spam, positive = spam):

- **FP:** email hợp lệ bị dự đoán là spam.
- **FN:** email spam bị dự đoán là hợp lệ.

Bài 4 — Accuracy trap

Model dự đoán mọi mẫu là negative (950 negative, 50 positive).

Confusion matrix:

	Dự đoán positive	Dự đoán negative
Thực tế positive	TP = 0	FN = 50
Thực tế negative	FP = 0	TN = 950

- Accuracy = (TP + TN) / 1000 = 950 / 1000 = **95%**
- Recall = TP / (TP + FN) = 0 / (0 + 50) = **0**
- Precision = TP / (TP + FP) = 0 / (0 + 0) = 0/0, **không xác định** (quy ước bằng 0 vì model không dự đoán positive nào)
- F1 = 2PR / (P + R) = 2·0·0 / (0 + 0), cũng không xác định (quy ước bằng **0** vì recall = 0)

Với lớp negative: Precision = 950/1000 = **0.95**, Recall = **1**, F1 = 2 × 0.95 × 1 / (0.95 + 1) ≈ **0.974**.

Metric	Giá trị	Nhận xét
Accuracy	0.95	Cao, nhưng chỉ vì lớp negative chiếm 95% dữ liệu
Recall (positive)	0	Không phát hiện được positive nào
Precision (positive)	0/0 (quy ước 0)	Không xác định
F1 (positive)	0	Model vô dụng với lớp positive

Kết luận: model **không tốt**. Accuracy cao chỉ vì dữ liệu mất cân bằng, còn recall và F1 của lớp positive đều bằng 0. Phải xem thêm recall, precision và F1 (theo từng lớp) chứ không chỉ accuracy.

Bài 1:

$x = [1, 2, 0]$ $w = [0.5, -0.2, 0.4]$

$b = 0,1$.

Tính $z = w^T x + b$

$w^T x = 0.5 \times 1 + (-0,2) \times 2 + 0,4 \times 0$

$= 0,5 + (-0,4)$

$= 0,1$

$\Rightarrow z = 0,1 + 0,1 = 0,2$

$\sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-0,2}} = 0,450$ ~~0,5498~~

$P(y=1|x)$

Vậy $P(y=1|x) = \sigma(z) \approx 0,5498$

Bài 2:

$P(y=1|x) = 0,72$ $T_1 = 0,5$ $T_2 = 0,8$

$T_1 = 0,5 \Rightarrow 0,72 > T_1$ nên dự đoán $\hat{y} = 1$

$T_2 = 0,8 \Rightarrow 0,72 < T_2$ nên dự đoán $\hat{y} = 0$

Ảnh hưởng của threshold:

- Tăng T → dự đoán positive giảm nhưng khó sai, FP giảm (precision tăng), FN tăng (vì có nhiều cái sác xuất cao nhưng không vượt qua T, bị dự đoán negative), recall giảm.
- Giảm T: model dự đoán positive dễ hơn → recall tăng FP tăng, precision giảm.

⇒ T tăng precision tăng, T giảm recall tăng.

HONG HA

Scanned with CamScanner

LAB 04 — Text Classification (IMDb)

Thứ ngày

Bài 3

Confusion matrix:

TP = 80 FN = 20

FP = 10 TN = 90

$precision = \frac{TP}{TP+FP} = \frac{80}{80+10} = \frac{8}{9} \approx 0,899$

$recall = \frac{TP}{TP+FN} = \frac{80}{80+20} = 0,8$

$F_1 = \frac{2PR}{P+R} = \frac{2.(8/9).(8/10)}{0,8+8/9} = \frac{0,8421}{}$

Trả lời:

FP: là email hợp lệ bị dự đoán là spam

FN: là email spam bị dự đoán hợp lệ

Câu 4

Lập confusion metrix

	Đoán đúng	đoán sai
đúng	TP = 0	FN = 50
sai	FP = 0	TN = 950

$accuracy = \frac{950}{1000} \frac{TP+TN}{} = \frac{950}{1000} = 95\%$

$Recall = \frac{TP}{TP+FN} = \frac{0}{0+50} = 0$

$Precision = \frac{TP}{TP+FP} = \frac{0}{0+9500} = \frac{0}{0}$

$F_1 = \frac{2.P.R}{P+R} = \frac{2.0.0}{0+0} = 0$

Với lớp negative: Precision = 0,95; recall = 1, F_1 = 0,974

HONG HA

Scanned with CamScanner

LAB 04 — Text Classification (IMDb)

Thứ ngày . .

metric	value	NX
accuracy	0,95	tốt
recall (p)	0	Ko có positive nào
precision (p)	0/0	Ko xác định
F_1 (p)	0	model vô dụng với positive

Model không tốt, accuracy cao nhưng metric khác không có ý nghĩa.

Phải xem thêm recall, precision, F_1

Scanned with CamScanner

HONG HA

Prediction

Dataset: **IMDb Movie Reviews** (sentiment, binary: positive / negative; review dài, trung bình khoảng 230 từ, nhãn cân bằng 50/50). Pipeline: raw text → TF-IDF hoặc Average Word2Vec → Logistic Regression.

Các dự đoán dưới đây dựa trên đặc tính của hai representation (TF-IDF: sparse, lexical, giữ thông tin từng term; Average Word2Vec: dense, semantic, mất thứ tự từ, mỗi từ một vector).

Dự đoán chính: TF-IDF hay Average Word2Vec phù hợp hơn với IMDb?

Prediction: TF-IDF + Logistic Regression tốt hơn Average Word2Vec + Logistic Regression một cách rõ rệt.

Reason: Review dài chứa nhiều từ cảm xúc đặc trưng (*excellent, awful, waste*...). TF-IDF giữ nguyên từng từ nên Logistic Regression gán được trọng số riêng cho từng từ. Average Word2Vec lấy trung bình vector của hàng trăm từ, làm loãng tín hiệu của các từ cảm xúc giữa rất nhiều từ trung tính.

Confidence: Cao

Prediction 1 — TF-IDF + Logistic Regression có khả năng hoạt động tốt không?

Prediction: Có, hoạt động rất tốt (accuracy khoảng 88–90%).

Reason: Cảm xúc phần lớn được quyết định bởi các từ có cực tính rõ, nên bài toán gần như tách được tuyến tính trong không gian từ vựng. Logistic Regression có đủ năng lực và ít overfit khi có regularization.

Confidence: Vừa

Prediction 2 — Average Word2Vec có chắc chắn tốt hơn TF-IDF không?

Prediction: Không. Average Word2Vec chắc chắn kém hơn TF-IDF trên bài toán này.

Reason: Averaging làm mất thứ tự từ, mỗi từ chỉ có một vector cố định, và việc nén cả review thành 100 số làm mất quá nhiều thông tin so với TF-IDF giữ từng term.

Confidence: Cao

Prediction 3 — Nếu tăng vocabulary size thì accuracy có chắc chắn tăng không?

Prediction: Không chắc chắn tăng mãi; accuracy tăng nhưng bão hòa dần.

Reason: Các từ thêm vào ngày càng hiếm, mang ít thông tin tổng quát nên lợi ích giảm dần. Không có chuyện tăng vocabulary mãi thì kết quả tốt mãi.

Confidence: Vừa

Prediction 4 — Nếu bỏ stopwords thì performance có chắc chắn tăng không?

Prediction: Không chắc chắn tăng; có thể giảm.

Reason: Danh sách stopwords chuẩn chứa các từ như *not, no, never, very*, tức là chính các từ đảo hoặc nhấn mạnh cảm xúc. Ngoài ra ảnh hưởng của việc bỏ stopwords có thể phụ thuộc vào chủ đề/bài toán (ví dụ sentiment cần từ phủ định, còn topic classification có thể ít bị ảnh hưởng hơn).

Confidence: Thấp

Prediction 5 — Accuracy và F1 có thể cho kết luận khác nhau không?

Prediction: Có thể. Trên IMDb thì accuracy và F1 gần như trùng nhau (vì dữ liệu cân bằng 50/50), nhưng khi dữ liệu mất cân bằng thì hai chỉ số có thể cho kết luận khác nhau.

Reason: IMDb cân bằng nên cả hai chỉ số dịch chuyển cùng nhau. Khi mất cân bằng, accuracy bị lớp đa số chi phối còn F1 thì không. Ví dụ ở Bài 4 trong `calculations.md`: model đoán mọi mẫu là negative (950 negative, 50 positive) có accuracy 95% nhưng F1 của lớp positive bằng 0.

Confidence: Cao

LAB 04 — Text Classification trên IMDb

Pipeline: `raw text → preprocessing → representation (TF-IDF | Avg Word2Vec) → classifier → evaluation → error analysis`.

Quy ước chống data leakage: mọi thành phần có *fit* (TfidfVectorizer, StandardScaler, Word2Vec, classifier) chỉ được fit trên **train**; validation dùng để chọn siêu tham số `C`; **test** chỉ dùng để báo cáo kết quả cuối.

```
import warnings, time, json
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, precision_recall_fscore_support)
from gensim.models import Word2Vec

import preprocessing as P

np.random.seed(P.SEED)
sns.set_theme(style="whitegrid")
rows = []   # gom kết quả -> results.csv

def metrics(y, pred):
    return dict(accuracy=accuracy_score(y, pred),
                precision=precision_score(y, pred, zero_division=0),
                recall=recall_score(y, pred, zero_division=0),
                f1=f1_score(y, pred, zero_division=0))

def log(experiment, representation, classifier, split, size, t, m):
    rows.append(dict(experiment=experiment, representation=representation, classifier=classifier,
                     split=split, feature_dim=size, train_time_s=round(t, 3),
                     **{k: round(v, 4) for k, v in m.items()}))
```

1. Experiment 1 — Dataset inspection (chưa train gì)

```
train_raw, test_raw = P.load_imdb()
print("train:", train_raw.shape, "| test:", test_raw.shape)
print("Số classes:", sorted(train_raw.label.unique()), "(0 = negative, 1 = positive)")

for name, df in [("train", train_raw), ("test", test_raw)]:
    print(f"\n[{name}] missing text: {df.text.isna().sum()} | empty text: {(df.text.str.strip()=='').sum()} |
missing label: {df.label.isna().sum()}")
    print(f"[{name}] duplicate texts: {df.text.duplicated().sum()}")

dist = []
for name, df in [("train", train_raw), ("test", test_raw)]:
    vc = df.label.value_counts().sort_index()
    for lab, cnt in vc.items():
        dist.append(dict(split=name, class_=("negative" if lab == 0 else "positive"), count=cnt,
percentage=round(100*cnt/len(df), 2)))
dist = pd.DataFrame(dist); display(dist)

train_raw["n_words"] = train_raw.text.str.split().str.len()
```

```python
print("\nĐộ dài (số từ) train:"); display(train_raw.n_words.describe().round(1))

fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
sns.barplot(data=dist, x="split", y="count", hue="class_", ax=ax[0])
ax[0].set_title("Class distribution (IMDb)")
sns.histplot(train_raw.n_words, bins=60, ax=ax[1], color="#4c72b0")
ax[1].set_title("Document length (words) – train"); ax[1].set_xlabel("words")
plt.tight_layout(); plt.savefig("class_distribution.png", dpi=150); plt.show()

overlap = len(set(train_raw.text) & set(test_raw.text))
print("Số review xuất hiện ở cả train lẫn test:", overlap)
```

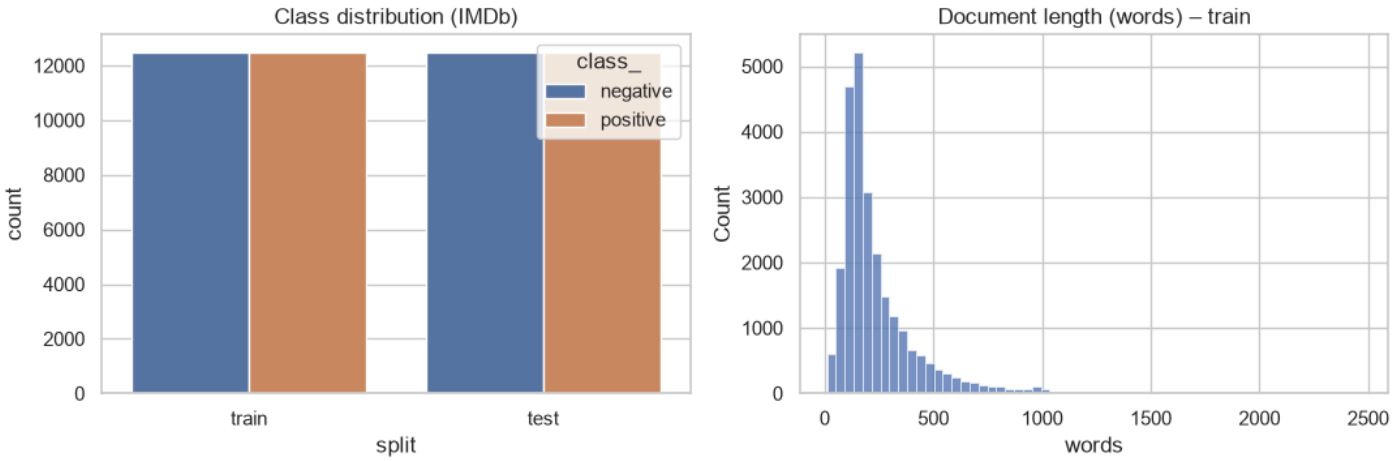
```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
train: (25000, 2) | test: (25000, 2)
Số classes: [np.int64(0), np.int64(1)] (0 = negative, 1 = positive)

[train] missing text: 0 | empty text: 0 | missing label: 0
[train] duplicate texts: 96

[test] missing text: 0 | empty text: 0 | missing label: 0
[test] duplicate texts: 199
```

	split	class_	count	percentage
0	train	negative	12500	50.0
1	train	positive	12500	50.0
2	test	negative	12500	50.0
3	test	positive	12500	50.0

```
Độ dài (số từ) train:
count    25000.0
mean       233.8
std        173.7
min         10.0
25%        127.0
50%        174.0
75%        284.0
max       2470.0
Name: n_words, dtype: float64
```




```
Số review xuất hiện ở cả train lẫn test: 123
```

Nhận xét: dataset gần như cân bằng hoàn toàn (50/50) ⇒ *không bị class imbalance*; do đó accuracy và F1 ở đây sẽ gần nhau (khác với bài tập "accuracy trap"). Có một số review trùng lặp trong train → loại bỏ trước khi split để cùng một review không rơi vào cả train lẫn validation.

2. Train / Validation / Test split + Data leakage

```
train_dedup = train_raw.drop_duplicates(subset="text").drop(columns="n_words").reset_index(drop=True)
print(f"Loại {len(train_raw)-len(train_dedup)} review trùng trong train: {len(train_raw)} -> {len(train_dedup)}")

train_df, val_df = P.make_splits(train_dedup, val_size=0.2)
test_df = test_raw.reset_index(drop=True)
for n, d in [("train", train_df), ("validation", val_df), ("test", test_df)]:
    print(f"{n:10s}: {len(d):6d} mẫu | positive = {d.label.mean():.3f}")

# chuẩn bị văn bản đã clean + token (cho Word2Vec)
for d in (train_df, val_df, test_df):
    d["clean"] = d.text.map(P.clean_text)
    d["tokens"] = d.clean.str.split()
```

```
Loại 96 review trùng trong train: 25000 -> 24904
train     :  19923 mẫu | positive = 0.501
validation:   4981 mẫu | positive = 0.501
test      :  25000 mẫu | positive = 0.500
```

Demo data leakage

Fit TF-IDF **trên toàn bộ dữ liệu** rồi mới split là sai: vocabulary và IDF đã "nhìn thấy" test. Dưới đây so sánh số term chỉ xuất hiện ở test (mà pipeline đúng sẽ coi là *unknown / bị bỏ qua*) với vocabulary khi fit sai.

```
vec_train_only = TfidfVectorizer(min_df=1).fit(train_df.clean)
vec_all = TfidfVectorizer(min_df=1).fit(pd.concat([train_df.clean, val_df.clean, test_df.clean]))
v_train, v_all = set(vec_train_only.vocabulary_), set(vec_all.vocabulary_)
print("Vocab (fit đúng: chỉ train):      ", len(v_train))
print("Vocab (fit sai: train+val+test): ", len(v_all))
print("Term CHỈ có ở val/test bị 'lộ' vào vocab khi fit sai:", len(v_all - v_train))

idf_t = dict(zip(vec_train_only.get_feature_names_out(), vec_train_only.idf_))
idf_a = dict(zip(vec_all.get_feature_names_out(), vec_all.idf_))
diff = np.array([abs(idf_t[w]-idf_a[w]) for w in v_train])
print(f"IDF của các term chung bị thay đổi: trung bình {diff.mean():.4f}, tối đa {diff.max():.4f}")
```

```
Vocab (fit đúng: chỉ train):       68036
Vocab (fit sai: train+val+test):  101213
Term CHỈ có ở val/test bị 'lộ' vào vocab khi fit sai: 33177
IDF của các term chung bị thay đổi: trung bình 0.4762, tối đa 2.2173
```

Vì sao là leakage: IDF của mỗi term được tính từ tần suất xuất hiện trên toàn corpus; nếu corpus gồm cả test thì thống kê của test đã "chảy" vào feature vector dùng để train/đánh giá. Test không còn là dữ liệu chưa từng thấy → kết quả đánh giá bị lạc quan, không phản ánh khả năng tổng quát hóa. Ở bên dưới mọi `fit` đều chỉ trên train.

3. Experiment 2 — Baseline: TF-IDF + Logistic Regression

Chọn `C` trên validation, sau đó báo cáo trên cả validation và test.

```
def make_tfidf(**kw):
    params = dict(preprocessor=None, min_df=2, sublinear_tf=True)
    params.update(kw)
    return TfidfVectorizer(**params)

def run_tfidf_lr(ngram=(1,1), C_grid=(0.1, 1, 10, 100), clf="lr", **vec_kw):
    """Fit TF-IDF trên train -> transform val/test -> chọn C trên val -> trả về mọi thứ."""
    vec = make_tfidf(ngram_range=ngram, **vec_kw)
    t0 = time.perf_counter()
    Xtr = vec.fit_transform(train_df.clean)
    t_vec = time.perf_counter() - t0
    Xva, Xte = vec.transform(val_df.clean), vec.transform(test_df.clean)
    best = None
    for C in C_grid:
        model = (LogisticRegression(C=C, max_iter=2000, solver="liblinear") if clf == "lr"
```

```
                else LinearSVC(C=C, max_iter=5000))
        t0 = time.perf_counter(); model.fit(Xtr, train_df.label); t_fit = time.perf_counter() - t0
        f1v = f1_score(val_df.label, model.predict(Xva))
        if best is None or f1v > best["val_f1"]:
            best = dict(C=C, model=model, val_f1=f1v, t_train=t_vec + t_fit)
    best.update(vec=vec, Xtr=Xtr, Xva=Xva, Xte=Xte, vocab=len(vec.vocabulary_))
    return best

base = run_tfidf_lr()
print(f"Best C = {base['C']} | vocab size = {base['vocab']} | training time (vectorize + fit) =
{base['t_train']:.2f}s")
for split, X, y in [("val", base["Xva"], val_df.label), ("test", base["Xte"], test_df.label)]:
    m = metrics(y, base["model"].predict(X)); print(split, {k: round(v,4) for k,v in m.items()})
    log("baseline", "TF-IDF (unigram)", "LogisticRegression", split, base["vocab"], base["t_train"], m)
```

```
Best C = 10 | vocab size = 40790 | training time (vectorize + fit) = 3.13s
val {'accuracy': 0.8906, 'precision': 0.8824, 'recall': 0.9018, 'f1': 0.892}
test {'accuracy': 0.8814, 'precision': 0.8842, 'recall': 0.8778, 'f1': 0.881}
```

4. Experiment 3 — So sánh representation (Model A / B / C)

- **A:** TF-IDF + Logistic Regression (baseline ở trên)
- **B:** Average Word2Vec + Logistic Regression — Word2Vec (CBOW, 100 chiều) train **chỉ trên văn bản train** (không nhìn val/test, không dùng nhãn)
- **C:** TF-IDF + Linear SVM

```
DIM = 100
t0 = time.perf_counter()
w2v = Word2Vec(sentences=train_df.tokens.tolist(), vector_size=DIM, window=5, min_count=2,
               workers=1, epochs=10, seed=P.SEED)
t_w2v = time.perf_counter() - t0
wv = w2v.wv
print(f"Word2Vec vocab = {len(wv.key_to_index)}, train time = {t_w2v:.1f}s")
print("Từ gần 'good':", [w for w, _ in wv.most_similar("good", topn=8)])
print("Từ gần 'bad' :", [w for w, _ in wv.most_similar("bad", topn=8)])
print("cos(good, bad) =", round(float(wv.similarity("good", "bad")), 3))

Xtr_w = P.docs_to_matrix(train_df.tokens, wv, DIM)
Xva_w = P.docs_to_matrix(val_df.tokens, wv, DIM)
Xte_w = P.docs_to_matrix(test_df.tokens, wv, DIM)
scaler = StandardScaler().fit(Xtr_w)              # fit trên TRAIN
Xtr_w, Xva_w, Xte_w = scaler.transform(Xtr_w), scaler.transform(Xva_w), scaler.transform(Xte_w)

best_w = None
for C in (0.01, 0.1, 1, 10):
    t0 = time.perf_counter()
    m_ = LogisticRegression(C=C, max_iter=3000).fit(Xtr_w, train_df.label)
    t_fit = time.perf_counter() - t0
    f1v = f1_score(val_df.label, m_.predict(Xva_w))
    if best_w is None or f1v > best_w[0]: best_w = (f1v, C, m_, t_fit)
_, C_w, model_w, t_fit_w = best_w
t_w_total = t_w2v + t_fit_w
print(f"Best C = {C_w} | tổng thời gian (Word2Vec + LR) = {t_w_total:.1f}s")
for split, X, y in [("val", Xva_w, val_df.label), ("test", Xte_w, test_df.label)]:
    m = metrics(y, model_w.predict(X)); print(split, {k: round(v,4) for k,v in m.items()})
    log("representation", "Avg Word2Vec", "LogisticRegression", split, DIM, t_w_total, m)
```

```
Word2Vec vocab = 46899, train time = 51.8s
Từ gần 'good': ['great', 'decent', 'bad', 'cool', 'nice', 'fine', 'ok', 'interesting']
Từ gần 'bad' : ['terrible', 'awful', 'good', 'horrible', 'stupid', 'lame', 'cheesy', 'poor']
cos(good, bad) = 0.722
Best C = 0.01 | tổng thời gian (Word2Vec + LR) = 51.9s
val {'accuracy': 0.8416, 'precision': 0.8353, 'recall': 0.8517, 'f1': 0.8434}
test {'accuracy': 0.8426, 'precision': 0.8435, 'recall': 0.8413, 'f1': 0.8424}
```

```
svm = run_tfidf_lr(clf="svm", C_grid=(0.01, 0.1, 1, 10))
print(f"Best C = {svm['C']} | vocab = {svm['vocab']} | time = {svm['t_train']:.2f}s")
for split, X, y in [("val", svm["Xva"], val_df.label), ("test", svm["Xte"], test_df.label)]:
    m = metrics(y, svm["model"].predict(X)); print(split, {k: round(v,4) for k,v in m.items()})
    log("representation", "TF-IDF (unigram)", "LinearSVC", split, svm["vocab"], svm["t_train"], m)

res = pd.DataFrame(rows)
tbl = res[res.split == "test"].query("experiment in ['baseline','representation']")[
    ["representation", "classifier", "feature_dim", "train_time_s", "accuracy", "precision", "recall", "f1"]]
print("\n=== KẾT QUẢ TRÊN TEST ==="); display(tbl.reset_index(drop=True))
```

```
Best C = 0.1 | vocab = 40790 | time = 2.22s
val {'accuracy': 0.8916, 'precision': 0.8811, 'recall': 0.9058, 'f1': 0.8933}
test {'accuracy': 0.8886, 'precision': 0.8862, 'recall': 0.8917, 'f1': 0.8889}

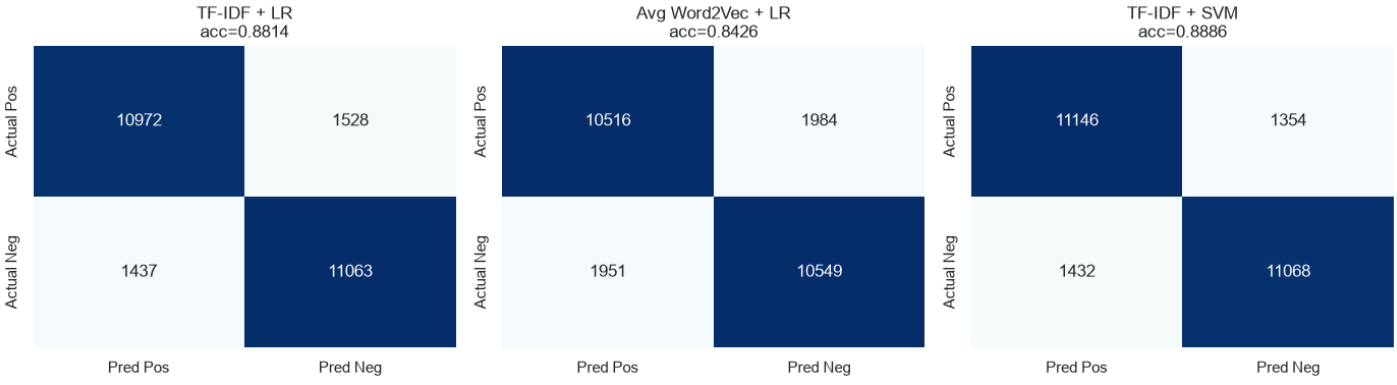
=== KẾT QUẢ TRÊN TEST ===
```

	representation	classifier	feature_dim	train_time_s	accuracy	precision	recall	f1
0	TF-IDF (unigram)	LogisticRegression	40790	3.128	0.8814	0.8842	0.8778	0.8810
1	Avg Word2Vec	LogisticRegression	100	51.911	0.8426	0.8435	0.8413	0.8424
2	TF-IDF (unigram)	LinearSVC	40790	2.218	0.8886	0.8862	0.8917	0.8889

5. Evaluation — Confusion matrix

```
preds_test = {
    "TF-IDF + LR":      base["model"].predict(base["Xte"]),
    "Avg Word2Vec + LR": model_w.predict(Xte_w),
    "TF-IDF + SVM":     svm["model"].predict(svm["Xte"]),
}
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, (name, pr) in zip(axes, preds_test.items()):
    cm = confusion_matrix(test_df.label, pr, labels=[1, 0])        # hàng/cột: Pos, Neg
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["Pred Pos", "Pred Neg"], yticklabels=["Actual Pos", "Actual Neg"])
    acc = accuracy_score(test_df.label, pr)
    ax.set_title(f"{name}\nacc={acc:.4f}")
    print(name, "| TP, FN, FP, TN =", cm.ravel().tolist())
plt.tight_layout(); plt.savefig("confusion_matrix.png", dpi=150); plt.show()
```

```
TF-IDF + LR | TP, FN, FP, TN = [10972, 1528, 1437, 11063]
Avg Word2Vec + LR | TP, FN, FP, TN = [10516, 1984, 1951, 10549]
TF-IDF + SVM | TP, FN, FP, TN = [11146, 1354, 1432, 11068]
```




```
y_te = test_df.label.values
for name, pr in preds_test.items():
    tn, fp, fn, tp = confusion_matrix(y_te, pr).ravel()
    print(f"{name:20s} FP={fp:5d} (neg->pos)  FN={fn:5d} (pos->neg)  | precision(pos)={tp/(tp+fp):.4f}
recall(pos)={tp/(tp+fn):.4f}")
```

```
TF-IDF + LR          FP= 1437 (neg->pos)  FN= 1528 (pos->neg)  | precision(pos)=0.8842 recall(pos)=0.8778
Avg Word2Vec + LR    FP= 1951 (neg->pos)  FN= 1984 (pos->neg)  | precision(pos)=0.8435 recall(pos)=0.8413
TF-IDF + SVM         FP= 1432 (neg->pos)  FN= 1354 (pos->neg)  | precision(pos)=0.8862 recall(pos)=0.8917
```

Ảnh hưởng của threshold (liên hệ Bài 2) và accuracy trap (liên hệ Bài 4)

```
proba_te = base["model"].predict_proba(base["Xte"])[:, 1]
th_rows = []
for tau in (0.3, 0.5, 0.7, 0.8, 0.9):
    p = (proba_te >= tau).astype(int)
    th_rows.append(dict(threshold=tau, **{k: round(v, 4) for k, v in metrics(y_te, p).items()}))
print("TF-IDF + LR trên test, đổi threshold:"); display(pd.DataFrame(th_rows))

# Accuracy trap: lấy tập test mất cân bằng 95% neg / 5% pos
rng = np.random.RandomState(P.SEED)
neg_idx = np.where(y_te == 0)[0]; pos_idx = np.where(y_te == 1)[0]
idx = np.concatenate([rng.choice(neg_idx, 950, replace=False), rng.choice(pos_idx, 50, replace=False)])
dummy = DummyClassifier(strategy="most_frequent").fit(base["Xtr"], train_df.label.map(lambda v: 0))
rows_trap = []
for name, pr in [("always negative", np.zeros(len(idx), dtype=int)), ("TF-IDF + LR", preds_test["TF-IDF + LR"]
[idx])]:
    rows_trap.append(dict(model=name, **{k: round(v, 4) for k, v in metrics(y_te[idx], pr).items()}))
print("\nTập test mất cân bằng 950 neg / 50 pos:"); display(pd.DataFrame(rows_trap))
```

```
TF-IDF + LR trên test, đổi threshold:
```

	threshold	accuracy	precision	recall	f1
0	0.3	0.8682	0.8206	0.9424	0.8773
1	0.5	0.8814	0.8842	0.8778	0.8810
2	0.7	0.8620	0.9330	0.7799	0.8496
3	0.8	0.8306	0.9522	0.6962	0.8043
4	0.9	0.7686	0.9716	0.5534	0.7051

```
Tập test mất cân bằng 950 neg / 50 pos:
```

	model	accuracy	precision	recall	f1
0	always negative	0.950	0.0000	0.0	0.0000
1	TF-IDF + LR	0.887	0.2797	0.8	0.4145

6. Các thí nghiệm kiểm chứng prediction: vocabulary size & stopwords

So sánh trên **validation** (không chạm test), TF-IDF unigram + LR với `C` cố định = best C của baseline.

```
def quick(vec_kw, ngram=(1,1)):
    vec = make_tfidf(ngram_range=ngram, **vec_kw)
    t0 = time.perf_counter(); Xtr = vec.fit_transform(train_df.clean)
    m_ = LogisticRegression(C=base["C"], max_iter=2000, solver="liblinear").fit(Xtr, train_df.label)
    t = time.perf_counter() - t0
    mv = metrics(val_df.label, m_.predict(vec.transform(val_df.clean)))
    return len(vec.vocabulary_), t, mv

out = []
for mf in (1000, 5000, 20000, None):
    v, t, mv = quick(dict(max_features=mf))
    out.append(dict(setting=f"max_features={mf}", vocab=v, time_s=round(t,2), val_acc=round(mv["accuracy"],4),
val_f1=round(mv["f1"],4)))
    log("vocab_size", f"TF-IDF max_features={mf}", "LogisticRegression", "val", v, t, mv)
v, t, mv = quick(dict(stop_words="english"))
out.append(dict(setting="remove stopwords (sklearn english)", vocab=v, time_s=round(t,2),
val_acc=round(mv["accuracy"],4), val_f1=round(mv["f1"],4)))
log("stopwords", "TF-IDF stopwords removed", "LogisticRegression", "val", v, t, mv)
display(pd.DataFrame(out))
print("Các từ phủ định trong stopword list của sklearn:", sorted(w for w in ENGLISH_STOP_WORDS if w in
{"not","no","nor","never","none","nothing","cannot","neither","nobody","very","too"}))
```

	setting	vocab	time_s	val_acc	val_f1
0	max_features=1000	1000	2.44	0.8595	0.8614
1	max_features=5000	5000	2.76	0.8753	0.8765
2	max_features=20000	20000	2.98	0.8878	0.8893
3	max_features=None	40790	2.93	0.8906	0.8920
4	remove stopwords (sklearn english)	40482	2.36	0.8822	0.8841

```
Các từ phủ định trong stopword list của sklearn: ['cannot', 'neither', 'never', 'no', 'nobody', 'none', 'nor',
'not', 'nothing', 'too', 'very']
```

7. Experiment 4 — Word order: "this movie is good" vs "this movie is not good"

```
pairs = [("this movie is good", "this movie is not good"),
         ("i like this movie", "i do not like this movie"),
         ("the plot is great", "the plot is not great"),
         ("a good film", "not a good film")]

# model bigram (để so sánh): TF-IDF 1-2gram + LR
bi = run_tfidf_lr(ngram=(1,2))

def vec_avg(s):  return P.average_embedding(P.tokenize(s), wv, DIM)
def cos(a, b):   return float(a @ b / (np.linalg.norm(a)*np.linalg.norm(b) + 1e-12))

rows4 = []
for s1, s2 in pairs:
    a, b = vec_avg(s1), vec_avg(s2)
    pw = lambda s: model_w.predict_proba(scaler.transform(vec_avg(s)[None]))[0, 1]
    pu = lambda s: base["model"].predict_proba(base["vec"].transform([s]))[0, 1]
    pb = lambda s: bi["model"].predict_proba(bi["vec"].transform([s]))[0, 1]
    rows4.append({"S1": s1, "S2": s2, "cos(avgW2V S1,S2)": round(cos(a, b), 3),
                  "P(pos) W2V S1": round(pw(s1),3), "P(pos) W2V S2": round(pw(s2),3),
                  "P(pos) TFIDF-uni S1": round(pu(s1),3), "P(pos) TFIDF-uni S2": round(pu(s2),3),
                  "P(pos) TFIDF-bi S1": round(pb(s1),3), "P(pos) TFIDF-bi S2": round(pb(s2),3)})
display(pd.DataFrame(rows4).T)

# bằng chứng định lượng: vector trung bình của S2 = (n1*vec(S1) + vec(not)) / (n1+1)
s1, s2 = pairs[0]
a, b = vec_avg(s1), vec_avg(s2)
n1 = len(P.tokenize(s1))
print("\n||avg(S2) - ((n1*avg(S1) + v_not)/(n1+1))|| =", float(np.linalg.norm(b - (n1*a + wv['not'])/(n1+1))), "(≈
0: 'not' chỉ được cộng vào vector rồi chia trung bình)")
print("Trọng số LR (unigram) của 'good':", round(float(base['model'].coef_[0]
[base['vec'].vocabulary_['good']]),3),
      "| 'not':", round(float(base['model'].coef_[0][base['vec'].vocabulary_['not']]),3))
vb = bi["vec"].vocabulary_
for g in ("not good", "very good", "not bad"):
    print(f"Trọng số LR (bigram) của '{g}':", round(float(bi['model'].coef_[0][vb[g]]),3) if g in vb else "n/a")
```

	0	1	2	3
S1	this movie is good	i like this movie	the plot is great	a good film
S2	this movie is not good	i do not like this movie	the plot is not great	not a good film
cos(avgW2V S1,S2)	0.948	0.922	0.914	0.923
P(pos) W2V S1	0.991	0.91	0.735	1.0
P(pos) W2V S2	0.851	0.043	0.234	0.998
P(pos) TFIDF-uni S1	0.934	0.19	0.996	0.988
P(pos) TFIDF-uni S2	0.503	0.02	0.961	0.759
P(pos) TFIDF-bi S1	0.997	0.625	1.0	0.999
P(pos) TFIDF-bi S2	0.003	0.475	0.843	0.278

```
||avg(S2) - ((n1*avg(S1) + v_not)/(n1+1))|| = 4.825388373319583e-07 (≈ 0: 'not' chỉ được cộng vào vector rồi chia
trung bình)
Trọng số LR (unigram) của 'good': 4.609 | 'not': -4.933
Trọng số LR (bigram) của 'not good': -6.472
Trọng số LR (bigram) của 'very good': 6.484
Trọng số LR (bigram) của 'not bad': 5.642
```

8. Experiment 5 — n-gram features: unigram vs unigram + bigram

```
rows5 = []
for name, ng, run in [("Unigram", (1,1), base), ("Uni + Bigram", (1,2), bi)]:
    pt = metrics(test_df.label, run["model"].predict(run["Xte"]))
    pv = metrics(val_df.label, run["model"].predict(run["Xva"]))
    rows5.append({"Features": name, "Vocabulary": run["vocab"], "Val F1": round(pv["f1"],4),
                  "Test F1": round(pt["f1"],4), "Test Acc": round(pt["accuracy"],4), "Training time (s)":
round(run["t_train"],2), "best C": run["C"]})
    log("ngram", f"TF-IDF {name}", "LogisticRegression", "test", run["vocab"], run["t_train"], pt)
    log("ngram", f"TF-IDF {name}", "LogisticRegression", "val", run["vocab"], run["t_train"], pv)
display(pd.DataFrame(rows5))
```

	Features	Vocabulary	Val F1	Test F1	Test Acc	Training time (s)	best C
0	Unigram	40790	0.8920	0.8810	0.8814	3.13	10
1	Uni + Bigram	364040	0.9041	0.9023	0.9022	9.32	100

9. Model interpretation — top features của Logistic Regression + TF-IDF

```
def top_features(run, k=20):
    names = np.array(run["vec"].get_feature_names_out()); w = run["model"].coef_[0]
    order = np.argsort(w)
    return (pd.DataFrame({"feature": names[order[::-1][:k]], "weight": w[order[::-1][:k]].round(3)}),
            pd.DataFrame({"feature": names[order[:k]], "weight": w[order[:k]].round(3)}))
pos_f, neg_f = top_features(base)
print("Top positive features (unigram):"); display(pos_f.T)
print("Top negative features (unigram):"); display(neg_f.T)
pos_b, neg_b = top_features(bi)
print("Top positive features (uni+bigram):"); display(pos_b.T)
print("Top negative features (uni+bigram):"); display(neg_b.T)

fig, ax = plt.subplots(1, 2, figsize=(11, 5))
ax[0].barh(pos_f.feature[::-1], pos_f.weight[::-1], color="#2a9d8f"); ax[0].set_title("Top positive (w >> 0)")
ax[1].barh(neg_f.feature[::-1], neg_f.weight[::-1], color="#d1495b"); ax[1].set_title("Top negative (w << 0)")
plt.tight_layout(); plt.savefig("top_features.png", dpi=150); plt.show()
```

```
Top positive features (unigram):
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	1
feature	great	excellent	perfect	wonderful	best	amazing	brilliant	superb	today	enjoyable	favorite	highly	subtle	rare	enjoye
weight	11.453	10.486	9.064	7.677	7.65	6.998	6.582	6.516	6.511	6.462	6.235	6.207	6.175	6.028	6.02

Top negative features (unigram):

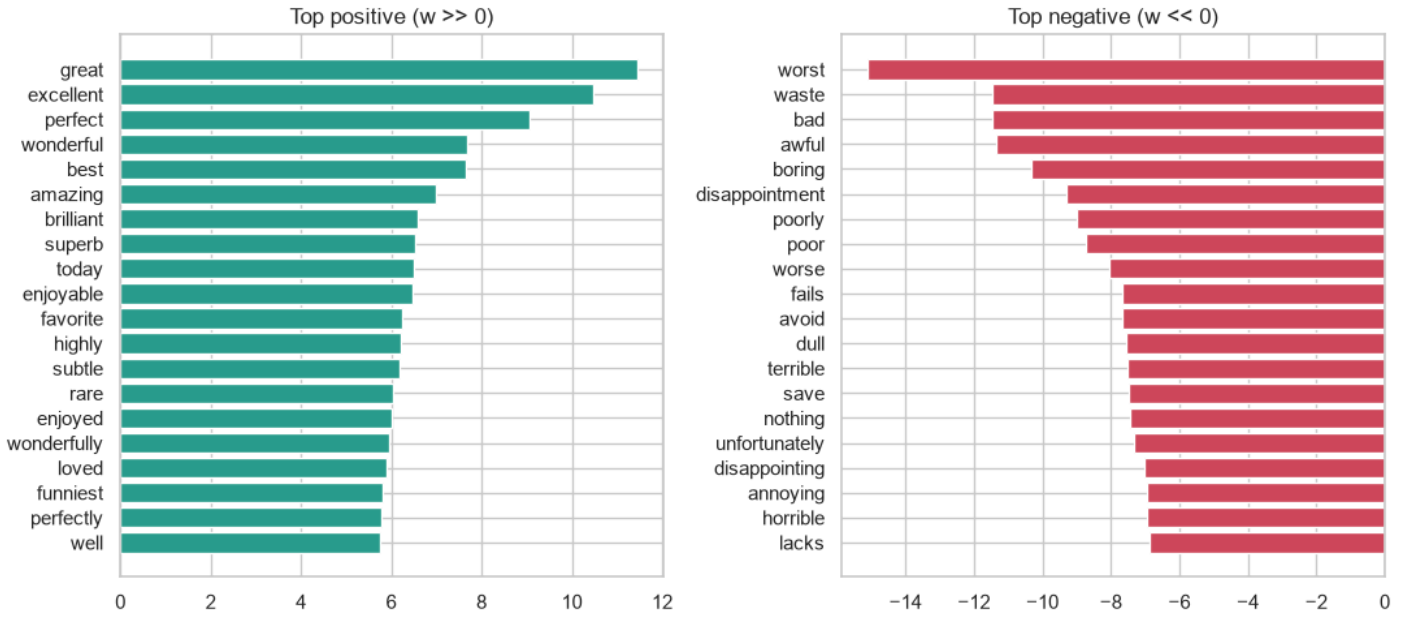
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
feature	worst	waste	bad	awful	boring	disappointment	poorly	poor	worse	fails	avoid	dull	terrible	save	noth
weight	-15.102	-11.477	-11.46	-11.361	-10.342	-9.3	-9.012	-8.738	-8.055	-7.684	-7.67	-7.564	-7.517	-7.484	-7.

Top positive features (uni+bigram):

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	1
feature	great	excellent	perfect	wonderful	amazing	the best	brilliant	fun	best	today	enjoyable	superb	love	loved	favorit
weight	20.537	15.934	13.558	13.14	11.321	10.866	10.239	10.095	9.936	9.871	9.681	9.403	9.346	9.225	9.16

Top negative features (uni+bigram):

	0	1	2	3	4	5	6	7	8	9	10	11	12	
feature	bad	worst	awful	boring	the worst	poor	waste	worse	poorly	terrible	disappointment	no	nothing	
weight	-21.672	-20.296	-17.819	-17.417	-15.655	-15.394	-14.392	-12.813	-12.493	-12.297	-12.258	-12.243	-12.2	-11



10. Error analysis — mẫu đúng/sai của baseline (TF-IDF + LR) trên test

```
pr_b = preds_test["TF-IDF + LR"]
test_df["pred"] = pr_b
test_df["p_pos"] = proba_te
test_df["n_words"] = test_df.clean.str.split().str.len()
test_df["correct"] = test_df.pred == test_df.label

errs = test_df[~test_df.correct]
print("Số lỗi:", len(errs), "| FP (neg->pos):", ((errs.label==0)).sum(), "| FN (pos->neg):",
((errs.label==1)).sum())
print("Độ dài trung bình (từ): đúng =", round(test_df[test_df.correct].n_words.mean(),1), "| sai =",
```

```
round(errs.n_words.mean(),1))

short = test_df[test_df.n_words <= 40]
print(f"\nReview ngắn (<=40 từ): {len(short)} mẫu, accuracy = {short.correct.mean():.4f}  vs toàn test
{test_df.correct.mean():.4f}")

# thống kê nhanh các hiện tượng gợi ý cho phân loại lỗi
neg_words = r"\b(?:not|no|never|n't|nothing|hardly|barely)\b|n't"
errs = errs.assign(has_negation=errs.clean.str.contains(neg_words, regex=True),
                   has_sarcasm_cue=errs.clean.str.contains(r"\b(?:oh great|yeah right|thanks for|wow just)\b",
regex=True))
print("Tỉ lệ lỗi có từ phủ định:", round(errs.has_negation.mean(), 3), "| tỉ lệ review nói chung có phủ định:",
round(test_df.clean.str.contains(neg_words, regex=True).mean(), 3))

def show(df, title, n=3, seed=P.SEED, maxlen=700):
    print("\n" + "="*20, title, "="*20)
    for i, r in df.sample(n, random_state=seed).iterrows():
        print(f"[idx={i}] true={r.label} pred={r.pred} P(pos)={r.p_pos:.3f} words={r.n_words}\n
{r.text[:maxlen]}\n")

mid = test_df[(test_df.n_words <= 120)]
show(mid[mid.correct & (mid.label == 1)], "ĐÚNG – positive", 2)
show(mid[mid.correct & (mid.label == 0)], "ĐÚNG – negative", 2)
show(mid[(~mid.correct) & (mid.label == 0)], "SAI – FP (thật negative, đoán positive)", 4)
show(mid[(~mid.correct) & (mid.label == 1)], "SAI – FN (thật positive, đoán negative)", 4)
```

```
Số lỗi: 2965 | FP (neg->pos): 1437 | FN (pos->neg): 1528
Độ dài trung bình (từ): đúng = 228.1 | sai = 230.5

Review ngắn (<=40 từ): 224 mẫu, accuracy = 0.8750  vs toàn test 0.8814
```

Tỉ lệ lỗi có từ phủ định: 0.883 | tỉ lệ review nói chung có phủ định: 0.882

========== ĐÚNG – positive ==========
[idx=18257] true=1 pred=1 P(pos)=0.967 words=90
  What gives this movie its personality is the knowledge, in the end, that it is all true. While it is a compelling and humorous documentary that does border on mockumentary in some parts (are they staged or is it really happening humorously?), it does seem to get a bit long towards the end.

This movie is funny in places it needs to be, and flows relatively well. Reminiscent of Christopher Guest's movies, if you liked those or Adaptation then I recommend you keep a look out for this one.

B+

[idx=19241] true=1 pred=1 P(pos)=0.957 words=19
  Without a doubt, one of Tobe Hoppor's best! Epic storytellng, great special effects, and The Spacegirl (vamp me baby!).

========== ĐÚNG – negative ==========
[idx=1796] true=0 pred=0 P(pos)=0.003 words=109
  I'm not sure why I disliked this film so much. Maybe I'm too old or too male or too something. Just who was the target audience here? If you're it and you liked it, then I'm happy for you. Personally, I found it a bit of a pill. The characters were uninteresting and unlikeable, the script was just plain embarrassing and some, though not all, of the acting was uninspired. Mawkish, tedious and occasionally nauseating -- Surely there's something better on.

On a related issue: Why is it that whenever I see Chris Klein in a film I get an urge to slap him silly? Does anyone else get that?

[idx=631] true=0 pred=0 P(pos)=0.001 words=58
  This is a terrible movie that only gets worse and seems to never end. The acting was bad, the plot was worse, and the special effects seemed to have been created by a 5th grade science class. Dennis Weaver is such a great actor and should have never taken such a part. My advise, DON'T WATCH THIS MOVIE!

========== SAI – FP (thật negative, đoán positive) ==========
[idx=1608] true=0 pred=1 P(pos)=0.926 words=72
  After seeing Forever Hollywood, it would be natural to want to see a John Waters film. At least, one get to say that they have joined the legions of cinema cognoscenti who have experienced the unique cinematic stylings of perhaps the best known non-mainstream director. It's worth the effort, and PF is a lot better than Eraserhead,and there is a certain campiness about his films which his followers find addicting.

[idx=5521] true=0 pred=1 P(pos)=0.599 words=106
  "Kicked in the Head" is all about the Corrigan character, a twenty something man on a quest to find himself, and his involvements with a handful of quirky characters. This thin and ambiguous story, which was written by Corrigan, has a make-it-up-as-you-go feel and a screenplay which smells like an uninspired low budget indie. In spite of that and some annoying Hindenburg scene interjections, the film has an off beat, quirky kind of charm which may appeal, in some small way, to people with a similar sense of humor. Not for everyone, not for most, but maybe fun for some. (D+)

[idx=9430] true=0 pred=1 P(pos)=0.526 words=118
  THANK GOD YOU'RE HERE is painful, positively painful. The title is apt, in a sense, if aimed at the large studio audience paid to laugh like they were watching the second coming of the Marx Brothers. And trust me, they are paid.

As creatively barren as the entertainment industry has become, I refuse to believe that NBC brass really have faith in this turkey. Rather, I think THANK GOD YOU'RE HERE is what all of you get who didn't watch, or didn't appreciate STUDIO 60, which previously graced the peacock network's Monday night lineup. You want to turn your nose up at caviar, fine. Here's some lovely Alpo direct from Menu Foods for you to slop around in.

[idx=6364] true=0 pred=1 P(pos)=0.596 words=78
  Dennis Quaid is tryin' hard to prove us that Jerry Lee Lewis was a dumb guy. And he's doing too much to prove it. TV sequences are very good, like a photocopy of old black and white footages. Music is fine too, because Mr. Lewis himself is singing. But the rest is just Hollywood B-Movie style, with the fifties Happy Days complex. I think the only good thing in this movie is to see young Winona Ryder.

========== SAI – FN (thật positive, đoán negative) ==========
[idx=21274] true=1 pred=0 P(pos)=0.457 words=119
  "Traffik 1989" is an Emmy award winning six part miniseries out of the UK which was the inspiration for the Oscar winning "Traffic 2000". The five hour film breaks down the opium/heroine trade for the viewer from the handcasting of poppy seeds in an Afghanistan field to the "head rush" of a mainlining junkie in a flat in England. Not only does "Traffik" offer entertainment value through interleaved dramatic stories it also provides an overview of the international drug trade at all levels answering the who, where, how, and why questions of the age

```
old and unstoppable narcotic supply/demand machine. Synergistically entertaining and educational, "Traffik" will prove to be time well spent for t

[idx=16946] true=1 pred=0 P(pos)=0.446 words=64
  The Coen's strike again. I had no presuppositions going in and I was amazed at the bizarre telling of a good-bad guy story. Although Clooney is easily replaceable in this, his cornball style is welcome. Turturro and Nelson are dead ringers. And I loved "I Am a Man of Constant Sorrow" as performed by the "Soggy Bottom Boys". Catchy tune... <br /><br />8 of 10

[idx=21966] true=1 pred=0 P(pos)=0.499 words=96
  I've seen this movie more than once. It isn't the greatest scifi flick I've every seen, but it is not a bad movie. The acting is good and the characters are more "real" than most in low budget sci fi. (At least it isn't full of dumb bimbos like so many other low budget scifi.) I especially like Elizabeth Pena. She is a good actress and she does worried single mother as well as any and better than some.<br /><br />Don't let the nay sayers run you off. See it for yourself and judge it for yourself.

[idx=12969] true=1 pred=0 P(pos)=0.011 words=55
  David Morse and Andre Braugher are very talented actors, which is why I'm trying so hard to support this program. Unfortunately, an irrational plot, and very poor writing is making it difficult for me. I'm hoping that the show gets a serious overhaul, or that the actors find new projects that are worthy of them.
```

11. Lưu kết quả

```
res = pd.DataFrame(rows)
res.to_csv("results.csv", index=False)
print(res.shape); display(res)
```

```
(15, 10)
```

	experiment	representation	classifier	split	feature_dim	train_time_s	accuracy	precision	recall	f1
0	baseline	TF-IDF (unigram)	LogisticRegression	val	40790	3.128	0.8906	0.8824	0.9018	0.8920
1	baseline	TF-IDF (unigram)	LogisticRegression	test	40790	3.128	0.8814	0.8842	0.8778	0.8810
2	representation	Avg Word2Vec	LogisticRegression	val	100	51.911	0.8416	0.8353	0.8517	0.8434
3	representation	Avg Word2Vec	LogisticRegression	test	100	51.911	0.8426	0.8435	0.8413	0.8424
4	representation	TF-IDF (unigram)	LinearSVC	val	40790	2.218	0.8916	0.8811	0.9058	0.8933
5	representation	TF-IDF (unigram)	LinearSVC	test	40790	2.218	0.8886	0.8862	0.8917	0.8889
6	vocab_size	TF-IDF max_features=1000	LogisticRegression	val	1000	2.436	0.8595	0.8513	0.8717	0.8614
7	vocab_size	TF-IDF max_features=5000	LogisticRegression	val	5000	2.757	0.8753	0.8698	0.8834	0.8765
8	vocab_size	TF-IDF max_features=20000	LogisticRegression	val	20000	2.979	0.8878	0.8787	0.9002	0.8893
9	vocab_size	TF-IDF max_features=None	LogisticRegression	val	40790	2.931	0.8906	0.8824	0.9018	0.8920
10	stopwords	TF-IDF stopwords removed	LogisticRegression	val	40482	2.362	0.8822	0.8715	0.8970	0.8841
11	ngram	TF-IDF Unigram	LogisticRegression	test	40790	3.128	0.8814	0.8842	0.8778	0.8810
12	ngram	TF-IDF Unigram	LogisticRegression	val	40790	3.128	0.8906	0.8824	0.9018	0.8920
13	ngram	TF-IDF Uni + Bigram	LogisticRegression	test	364040	9.316	0.9022	0.9019	0.9027	0.9023
14	ngram	TF-IDF Uni + Bigram	LogisticRegression	val	364040	9.316	0.9028	0.8942	0.9142	0.9041

Error analysis — TF-IDF (unigram) + Logistic Regression trên IMDb test

Model phân tích: baseline `TF-IDF + Logistic Regression`, accuracy test = 0.8814. Các mẫu lấy từ test set (mục 10 của `experiment.ipynb`); `idx` là chỉ số dòng trong test set.

1. Confusion matrix

	Pred Pos	Pred Neg
Actual Pos	TP = 10972	FN = 1528
Actual Neg	FP = 1437	TN = 11063

- Tổng lỗi = 2965; trong đó **FN (1528) nhiều hơn FP (1437)** một chút.
- Precision = 0.8842, Recall = 0.8778: recall thấp hơn precision, tức là model **bỏ sót positive nhiều hơn là báo nhầm positive**.
- Metric phản ánh vấn đề này là **recall** (và precision cho hướng ngược lại).

2. Mẫu phân loại ĐÚNG (3 mẫu)

idx	Text (rút gọn)	True	Predicted (P(pos))	Analysis
19241	"Without a doubt, one of Tobe Hoppor's best! Epic storytellng, great special effects..."	positive	positive (0.957)	Có nhiều từ cảm xúc mạnh (*best, epic, great*) nên dễ phân loại.
631	"This is a terrible movie that only gets worse... The acting was bad... Dennis Weaver is such a great actor... DON'T WATCH THIS MOVIE!"	negative	negative (0.001)	Tín hiệu âm (*terrible, worse, bad*) nhiều hơn tín hiệu dương (*great*), nên tổng trọng số nghiêng về âm.
1796	"I'm not sure why I disliked this film so much... uninteresting and unlikeable... embarrassing... tedious... nauseating"	negative	negative (0.003)	Rất nhiều từ âm mạnh bù đắp cho cụm tích cực giả định ("if you liked it, then I'm happy for you").

3. Mẫu phân loại SAI (4 mẫu)

idx	Text (rút gọn)	True	Predicted (P(pos))	Category	Analysis
21966	"It isn't the greatest scifi flick I've ever seen, but it is not a bad movie. The acting is good... Don't let the nay sayers run you off."	positive	negative (0.499)	**Negation**	"not a bad movie" thực ra là khen, nhưng túi từ thấy *not* và *bad* như hai feature riêng, và *bad* có trọng số âm mạnh.

idx	Text (rút gọn)	True	Predicted (P(pos))	Category	Analysis
9430	"THANK GOD YOU'RE HERE is painful, positively painful… this turkey… Here's some lovely Alpo… the audience paid to laugh…"	negative	positive (0.526)	**Lexical ambiguity**	*positively* (trạng từ nhấn mạnh) và *thank god* trông như từ tích cực.
21274	"'Traffik 1989'… breaks down the opium/heroine trade… a junkie… Synergistically entertaining and educational… time well spent."	positive	negative (0.457)	**Domain-specific term**	Các từ chỉ chủ đề (*opium, heroine, junkie*) mang trọng số âm dù không phải đánh giá về phim.
1608	"…It's worth the effort, and PF is a lot better than Eraserhead… the best known non-mainstream director…"	negative	positive (0.926)	**Data noise**	Nội dung đọc như khen nhưng nhãn là negative, nhãn có thể không nhất quán.

4. Tổng hợp

- Trong 4 mẫu sai có 4 nguyên nhân khác nhau: negation, lexical ambiguity, domain-specific term và data noise.
- Ba nguyên nhân đầu (negation, lexical ambiguity, domain-specific term) đều xuất phát từ việc TF-IDF coi mỗi từ là một feature độc lập, có một trọng số cố định bất kể ngữ cảnh.
- Nguyên nhân thứ tư (data noise) không phải lỗi của representation mà là nhãn không khớp với nội dung.

Reflection — từ representation đến neural NLP

Số liệu lấy từ `experiment.ipynb` / `results.csv` (IMDb, test 25 000 review).

Representation	Classifier	Accuracy	F1
TF-IDF (unigram)	Logistic Regression	0.8814	0.8810
Average Word2Vec	Logistic Regression	0.8426	0.8424
TF-IDF (unigram)	Linear SVM	0.8886	0.8889
TF-IDF (uni + bigram)	Logistic Regression	0.9022	0.9023

1. Vì sao TF-IDF (0.8814) khác Average Word2Vec (0.8426)?

- TF-IDF giữ nguyên từng từ nên Logistic Regression gán được trọng số riêng cho từ cảm xúc; Average Word2Vec lấy trung bình hàng trăm từ nên tín hiệu bị loãng.
- Word2Vec được tự train trên corpus nhỏ (khoảng 20 nghìn review) nên embedding chưa tốt.
- Word2Vec học theo ngữ cảnh nên *good* và *bad* có vector gần nhau (cosine 0.72 trong thí nghiệm), làm mất tính cực của từ.

2. Average embedding có hiểu "not" không? (Exp 4)

Với "this movie is good" → "this movie is not good": cosine giữa hai vector là 0.948; P(positive) của Word2Vec giảm từ 0.991 xuống 0.851, TF-IDF unigram từ 0.934 xuống 0.503, TF-IDF bigram từ 0.997 xuống 0.003.

→ **Hiểu một phần:** xác suất có giảm (0.991 → 0.851) nên "not" có tác động, nhưng không đảo được ý của câu.

3. Bigram giúp gì và tốn gì? (Exp 5)

Features	Vocabulary	F1	Training time
Unigram	40 790	0.8810	3.1 s
Uni + Bigram	364 040	0.9023	9.3 s

Bigram bắt được cụm đảo cực tính như *not good, not bad*; nhưng số cặp từ khác nhau rất nhiều nên vocabulary phình to, nhiều bigram hiếm dễ gây overfit.

4. Top features có ý nghĩa không?

Top positive: *great, excellent, perfect, wonderful…*; top negative: *worst, waste, bad, awful, boring…*. Phần lớn có nghĩa, nhưng vài từ (*today, save*) là tương quan tình cờ của tập train.

5. So prediction ban đầu với kết quả thật

Các dự đoán đều đúng hướng: TF-IDF tốt hơn Average Word2Vec (0.8814 so với 0.8426); tăng vocabulary thì F1 tăng nhưng bão hòa dần (0.861 → 0.877 → 0.889 → 0.892); bỏ stopwords làm F1 giảm (0.892 → 0.884); accuracy ≈ F1 trên IMDb vì dữ liệu cân bằng. Lý do là hiểu đúng đặc tính của hai representation (sparse/lexical và dense/mất thứ tự).

6. Giới hạn dẫn tới hướng nào ở LAB 05 trở đi?

- Cần mô hình đọc theo thứ tự (RNN/LSTM/GRU) để "not" ảnh hưởng được tới "good" ở sau.
- Cần dữ liệu lớn hơn hoặc embedding pretrained.
- Cần contextual embedding (một từ có nhiều vector tùy ngữ cảnh), rồi attention/Transformer.

7. Bài học rút ra

- Không chỉ nhìn accuracy; phải xem confusion matrix và precision/recall/F1.
- Representation quan trọng hơn lựa chọn classifier (Linear SVM 0.8886, Logistic Regression 0.8814, Average Word2Vec 0.8426).
- Phải phân tích lỗi mới biết representation hiện tại thiếu gì.